

第 6 章

在 UBUNTU 下使用 mysql

朱泽明

Mysql 安装

www.qianrushi.com.cn

❖ 安装 mysql 服务端

```
sudo apt-get install mysql-server
```

❖ 安装图形开发界面

```
sudo apt-get install mysql-workbench
```

❖ 安装 mysql 开发包

```
sudo apt-get install libmysqlclient-dev
```

头文件与库

www.qianrushi.com.cn

- ❖ 使用 MYSQL 函数必须包含 `<mysql/mysql.h>`
- ❖ 编译时必须指定库的位置和库名
 - L /usr/lib/mysql -lmysqlclient

❖ MYSQL

该结构代表 1 个数据库连接的句柄。几乎所有的 MySQL 函数均使用它。不应尝试拷贝 MYSQL 结构。不保证这类拷贝结果会有用

❖ MYSQL_RES

该结构代表返回行的查询结果（ SELECT, SHOW, DESCRIBE, EXPLAIN ）。在本节的剩余部分，将查询返回的信息称为“结果集”。

❖ MYSQL_ROW

这是 1 行数据的“类型安全”表示。它目前是按照计数字节字符串的数组实施的。（如果字段值可能包含二进制数据，不能将其当作由 Null 终结的字符串对待，这是因为这类值可能会包含 Null 字节）。行是通过调用 `mysql_fetch_row()` 获得的。

❖ MYSQL_FIELD

该结构包含关于字段的信息，如字段名、类型和大小。这里详细介绍了其成员。通过重复调用 `mysql_fetch_field()`，可为每个字段获得 `MYSQL_FIELD` 结构。字段值不是该结构的组成部份，它们包含在 `MYSQL_ROW` 结构中。

❖ MYSQL_FIELD_OFFSET

这是 MySQL 字段列表偏移量的“类型安全”表示（由 `mysql_field_seek()` 使用）。偏移量是行内的字段编号，从 0 开始。

mysql 常用 API

mysql_init()

www.qianrushi.com.cn

❖ MYSQL *mysql_init(MYSQL *mysql)

❖ 描述

分配或初始化与 mysql_real_connect() 相适应的 MYSQL 对象。如果 mysql 是 NULL 指针，该函数将分配、初始化、并返回新对象。否则，将初始化对象，并返回对象的地址。如果 mysql_init() 分配了新的对象，当调用 mysql_close() 来关闭连接时。将释放该对象。

❖ 返回值

初始化的 MYSQL* 句柄。如果无足够内存以分配新的对象，返回 NULL。

mysql_real_connect()

www.qianrushi.com.cn

❖ MySQL *mysql_real_connect(MYSQL *mysql, const char *host, const char *user, const char *passwd, const char *db, unsigned int port, const char *unix_socket, unsigned long client_flag)

❖ 描述

mysql_real_connect() 尝试与运行在主机上的 MySQL 数据库引擎建立连接。在你能够执行需要有效 MySQL 连接句柄结构的任何其他 API 函数之前，mysql_real_connect() 必须成功完成。关闭连接。

如果“port”不是 0，其值将用作 TCP/IP 连接的端口号。

如果 unix_socket 不是 NULL，该字符串描述了应使用的套接字或命名管道。注意，“host”参数决定了连接的类型。

client_flag 的值通常为 0。

❖ 返回值

如果连接成功，返回 MYSQL* 连接句柄。如果连接失败，返回 NULL。对于成功的连接，返回值与第 1 个参数的值相同。

mysql_real_query()

www.qianrushi.com.cn

❖ `int mysql_real_query(MYSQL *mysql, const char *query, unsigned long length)`

❖ 描述

执行由“query”指向的 SQL 查询，它应是字符串长度字节“long”。正常情况下，字符串必须包含 1 条 SQL 语句，而且不应为语句添加终结分号（‘;’）或“\g”。如果允许多语句执行，字符串可包含由分号隔开的多条语句。对于包含二进制数据的查询，必须使用 `mysql_real_query()` 而不是 `mysql_query()`。

❖ 返回值

如果查询成功，返回 0。如果出现错误，返回非 0 值。

mysql_store_result()

www.qianrushi.com.cn

❖ MYSQL_RES *mysql_store_result(MYSQL *mysql)

❖ 描述

对于成功检索了数据的每个查询（SELECT、SHOW、DESCRIBE、EXPLAIN、CHECK TABLE等），必须调用 mysql_store_result() 或 mysql_use_result()。mysql_store_result() 将查询的全部结果读取到客户端，分配 1 个 MYSQL_RES 结构，并将结果置于该结构中。调用 mysql_num_rows() 可以找出结果集中的行数。可以调用 mysql_fetch_row() 来获取结果集中的行，或调用 mysql_row_seek() 和 mysql_row_tell() 来获取或设置结果集中的当前行位置。

❖ 返回值

具有多个结果的 MYSQL_RES 结果集合。如果出现错误，返回 NULL。。

mysql_use_result()

www.qianrushi.com.cn

❖ MYSQL_RES *mysql_use_result(MYSQL *mysql)

❖ 描述

mysql_use_result() 将初始化结果集检索，但并不像 mysql_store_result() 那样将结果集实际读取到客户端。它必须通过对 mysql_fetch_row() 的调用，对每一行分别进行检索。这将直接从服务器读取结果，而不会将其保存在临时表或本地缓冲区内，与 mysql_store_result() 相比，速度更快而且使用的内存也更少。使用 mysql_use_result() 时，必须执行 mysql_fetch_row()，直至返回 NULL 值

❖ 返回值

具有多个结果的 MYSQL_RES 结果集合。如果出现错误，返回 NULL。。

二者比较

www.qianrushi.com.cn

- ❖ 调用 `mysql_store_result()`，一次性地检索整个结果集。该函数能从服务器获得查询返回的所有行，并将它们保存在客户端
- ❖ 通过调用 `mysql_use_result()`，对“按行”结果集检索进行初始化处理。该函数能初始化检索结果，但不能从服务器获得任何实际行。
- ❖ 在这两种情况下，均能通过调用 `mysql_fetch_row()` 访问行，必须都调用 `mysql_free_result()` 释放结果集使用的内存

- ❖ `mysql_store_result()` 的 1 个优点在于，由于将行全部提取到了客户端上，你不仅能连续访问行，还能使用 `mysql_data_seek()` 或 `mysql_row_seek()` 在结果集中向前或向后移动，以更改结果集内当前行的位置。通过调用 `mysql_num_rows()`，还能发现有多少行。另一方面，对于大的结果集，`mysql_store_result()` 所需的内存可能会很大，你很可能遇到内存溢出状况。

❖ `mysql_use_result()` 的 1 个优点在于，客户端所需的用于结果集的内存较少，原因在于，一次它仅维护一行（由于分配开销较低，`mysql_use_result()` 能更快）。它的缺点在于，你必须快速处理每一行以避免妨碍服务器，你不能随机访问结果集中的行（只能连续访问行），你不知道结果集中有多少行，直至全部检索了它们为止。不仅如此，即使在检索过程中你判定已找到所寻找的信息，也必须检索所有的行。

mysql_fetch_row()

www.qianrushi.com.cn

❖ MYSQL_ROW mysql_fetch_row(MYSQL_RES *result)

❖ 描述

检索结果集的下一行。如果行中保存了调用 mysql_fetch_row() 返回的值，将按照 row[0] 到 row[mysql_num_fields(result)-1]，访问这些值的指针

❖ 返回值

下一行的 MYSQL_ROW 结构。如果没有更多要检索的行或出现了错误，返回 NULL。

mysql_free_result()

www.qianrushi.com.cn

❖ void mysql_free_result(MYSQL_RES *result)

❖ 描述

释放由

mysql_store_result()、mysql_use_result()、mysql_list_dbs() 等为结果集分配的内存。完成对结果集的操作后，必须调用 mysql_free_result() 释放结果集使用的内存。

❖ 返回值
无。

mysql_error()

www.qianrushi.com.cn

❖ `const char *mysql_error(MYSQL *mysql)`

❖ 描述

对于由 `mysql` 指定的连接，对于失败的最近调用的 API 函数，`mysql_error()` 返回包含错误消息的、由 Null 终结的字符串。如果成功，所有向服务器请求信息的函数均会复位 `mysql_error()`。

❖ 返回值

返回描述错误的、由 Null 终结的字符串。如果未出现错误，返回空字符串。

mysql_close()

www.qianrushi.com.cn

❖ void mysql_close(MYSQL *mysql)

❖ 描述

关闭前面打开的连接。如果句柄是由 mysql_init() 或 mysql_connect() 自动分配的，mysql_close() 还将解除分配由 mysql 指向的连接句柄。

❖ 返回值
无。

mysql_commit()

www.qianrushi.com.cn

❖ `my_bool mysql_commit(MYSQL *mysql)`

❖ 描述

提交当前事务。

该函数的动作受 `completion_type` 系统变量的值控制。尤其是，如果 `completion_type` 的值为 2，终结事务并关闭客户端连接后，服务器将执行释放操作。客户端程序应调用 `mysql_close()`，从客户端一侧关闭连接。

❖ 返回值

如果成功，返回 0，如果出现错误，返回非 0 值。

示例代码

www.qianrushi.com.cn

```
#include<mysql/mysql.h>
```

```
MYSQL *conn;  
MYSQL_RES *res;  
MYSQL_ROW row;
```

```
char *server = "localhost";  
char *user = "root";  
char *password = ""; /* 此处改成你的密码 */  
char *database = "shangqian"; /* 预先建好的库名  
*/
```

// 初始化一个数据库句柄

```
conn = mysql_init(NULL);
```

// 连接数据库

```
if (!mysql_real_connect(conn, server, user,  
    password, database, 0, NULL, 0))  
{  
    fprintf(stderr, "%s\n", mysql_error(conn));  
    exit(1);  
}
```

// 设置字符集

```
mysql_set_character_set(conn, "utf8");
```

// 发送 SQL 语句，可以是 SELECT, INSERT, UPDATE 等各种 SQL 语句

```
if (mysql_query(conn, "select * from student"))  
{  
    fprintf(stderr, "%s\n", mysql_error(conn));  
    exit(1);  
}
```

// 获得结果集

```
res = mysql_use_result(conn);
```

// 依次从结果集中获得每一个行，并打印

```
while ((row = mysql_fetch_row(res)) != NULL)
```

```
{
```

```
    printf("%s \t%s\t%s\n", row[0], row[1], row[2]);
```

```
}
```

// 释放结果集

```
mysql_free_result(res);
```

// 关闭数据库连接

```
mysql_close(conn);
```


编译

www.qianrushi.com.cn

```
gcc test.c -o mysqltest  
-L /usr/lib/mysql -lmysqlclient
```

练习

www.qianrushi.com.cn

❖ 使用数据库代替链表，完成一个简易版学生管理系统

使用用户表、班级表、学生表、教师表

- 用户登录 (用户表中找用户名对应密码)
- 学生增删查改操作
- 教师增删查改操作
- 班级增删查改操作
- 建立班级、教师、学生之间的对应关系

❖ 支持以下功能

- 列出班级，根据班级查看本班所有学生
- 列出教师，查看某教师所带班级或所有学生
- 根据学号、姓名分别查询学生，可以重名